

# Viability and Compliance of the RIS System and Canonical Ledger Approach

## Overview of the Proposed System (RIS and Canonical Ledgers)

The **Referential Instruction Stub (RIS)** system is a method to modularize and reuse instruction content across multiple custom GPTs. In this approach, long or repetitive instructions (like persona definitions, tone guidelines, suffix rules, etc.) are stored in **canonical, read-only ledger files** instead of being written inline for each GPT <sup>1</sup>. Each GPT's instruction block then contains a short *stub* (alias) referencing the relevant file and section. At runtime, the GPT will *pull in* the content from those files, effectively activating the logic without consuming as much space in the prompt <sup>2</sup>. This design offloads persistent behaviors (tone, persona, style rules) into external files while preserving the main prompt's character budget <sup>3</sup>. The "canon" concept refers to treating these ledger files as **single sources of truth** – once finalized (canon-locked), their content is not meant to be altered except through deliberate version updates <sup>3</sup> <sup>4</sup>. This ensures consistency and prevents drift in the GPTs' behavior across updates.

Several canonical ledger files have been provided, each serving a specific role in the GPT's behavior architecture <sup>5</sup>. For example, `dataLedger_persona_v3.md` contains voice and tone archetypes (how the AI should "sound" and express itself), `dataLedger_system_v3.md` defines suffix rules and lifecycle or formatting conventions, and `dataLedger_parameters_v3.md` holds execution flags or toggles for behavior <sup>5</sup>. Others like `dataLedger_registry_v3.md` list the GPTs' identity metadata and capabilities, and optional files (hydration, narrative, ideation logs) handle advanced features like cross-session state carryover or world-building lore <sup>6</sup>. All these files collectively form the **canon** for the project – a cohesive set of rules and style guidelines that multiple GPT instances can reference.

**In summary:** The RIS system proposes using short pointers in a GPT's instructions to reference these canon files (e.g. a stub might point to a persona file section like `JoyWarden.Core` or a suffix enforcement section) <sup>2</sup>. This yields two key benefits: (1) **Technical efficiency** – saving prompt space since each stub might be ~30 characters instead of hundreds of characters of repeated text <sup>7</sup> <sup>8</sup>, and (2) **Consistency** – ensuring all GPTs share the same definitions for tone or behavior, since they all refer back to the same canon source <sup>3</sup>. The approach is intended for an ecosystem of GPTs (the "Glee-fully Personalizable Tools™" and **OverKill Hill P<sup>3</sup>** project) where many agents have shared personality traits or formatting rules <sup>9</sup>. RIS allows an update to one canon file to propagate across all related GPTs easily – update the file once, reattach it to each GPT, and all of them now exhibit the new behavior without individually editing their prompts <sup>10</sup>.

From a **compliance and viability perspective**, we need to verify that this idea aligns with OpenAI's current platform capabilities and policies. Key aspects to examine include: the ability to attach and reference external instruction files in ChatGPT (and whether this is officially supported), the use of the `@mention` system for multi-GPT handoffs, the content of these files with respect to OpenAI's usage policies (no disallowed content or behavior), and any technical limitations (like context size limits) that might affect this system. We will also outline an implementation plan, showing how to practically set up and use the RIS +

canon approach using the features available in ChatGPT as of mid-2025. Overall, the goal is to ensure **nothing in this proposal conflicts with OpenAI's system rules or technical constraints**, and that the plan is indeed *doable* with the current state of ChatGPT.

## Alignment with ChatGPT's Current Features and Limits

### Custom Instructions and Resource Files

OpenAI's ChatGPT (especially for Plus users) supports **Custom GPTs**, which allow a user to define a custom instruction prompt and even attach additional reference files. However, there is a known limit of **8,000 characters** for the instruction text field in a custom GPT (at least for GPT-4 models) <sup>11</sup> <sup>12</sup>. The RIS system was conceived specifically to work within this limit by moving large instruction blocks into external files <sup>3</sup>. This is fully in line with how advanced users currently extend custom GPT instructions. In fact, OpenAI's interface allows uploading *resource files* (up to 20 files, each up to 512 MB) to a custom GPT, which can serve as extended knowledge or instruction material <sup>12</sup> <sup>13</sup>. The approach described (storing persistent instructions in files and telling the GPT to use them) is essentially a supported workaround to the 8k character cap. As one expert noted, *"it's an 8,000 character limit in custom GPTs, but you can get around that by...upload[ing] a document with all of your instructions in it, and then within your instructions, just say, 'Use this document as your instructions.'"* <sup>12</sup>. In other words, OpenAI **intended** resource files to be used in exactly this fashion – to hold additional instructions or data that won't fit in the main prompt.

The RIS YAML stub structure (e.g. `referential_stubs: voice_profile: dataLedger_persona_v3.md::...`) is a formalized way to do this. While the exact YAML key `referential_stubs` is a project-specific convention, the concept it represents is supported. You would attach the file `dataLedger_persona_v3.md` to the custom GPT, then in the instructions field include a pointer or note to use that file's content. Since the ChatGPT UI might not automatically parse a custom YAML, in practice you might phrase it as an instruction like: *"Refer to the attached Persona file, section JoyWarden.Core, for the voice and personality guidelines."* This achieves the same end – the model will have the file in context (the system includes attached files in the model's accessible information) and will apply those instructions. **Viability:** There is no conflict here; using attached files to extend instructions is supported and common, and it preserves the character budget exactly as intended <sup>7</sup>. Each stub only occupies a short line in the prompt, but the GPT can access the full content of the file when generating responses <sup>7</sup>. As long as the files are attached properly, the model will follow the detailed tone/behavior rules in them.

One technical consideration is *how* the model accesses these files. Based on OpenAI's documentation, attached files are not automatically read in full for every response; rather, the model will consider their content when relevant (the exact mechanism might be by embedding the file content behind the scenes or via retrieval). The **instruction to use the file** is important. In testing, users often include a line like *"Here is additional context in [file name]; use it when responding."* The RIS stub serves as that indicator. So it's important in implementation to ensure the stub or an equivalent instruction explicitly tells the model to load/apply the file content. The example given in the attachments shows exactly that pattern: the stub instructs the GPT to pull in those traits at runtime <sup>2</sup>. This means the idea is compliant with how ChatGPT expects developers to reference resource files – it's essentially a built-in feature, not a hack. OpenAI's interface even lists the attached files in the GPT's public schema (in the `KnowledgeAssets` section, we see `files: []` which would list any attached file names) <sup>14</sup>. Thus, using **canonical ledger files** for persona, style, etc., is **viable** and does not violate any OpenAI rules. It leverages the platform's functionality as intended.

## Multi-GPT Collaboration via @mention

The project envisions multiple GPT “agents” (Toolbox, Tools, Tool-ettes, etc.) working together and handing off conversations. OpenAI **currently supports multi-GPT interactions** in a single chat through the **@mentions** feature. As of 2025, a user (or developer) can type @ in the chat to summon a list of their custom GPTs, and direct the next message to one of those GPT personas <sup>15</sup>. The conversation’s context (chat history) is retained across these switches <sup>16</sup>, meaning GPT-B will see what GPT-A and the user have said so far, but GPT-B will use its own attached files/instructions going forward. This exactly matches the RIS design: *“When GPT-A calls GPT-B using @mention, the active thread carries its current runtime context — but the files remain those of GPT-B.”* <sup>17</sup>. In other words, ChatGPT will maintain the running conversation, and simply swap in GPT-B’s profile (system prompt + attached canon files) when you @mention GPT-B.

**Viability:** The RIS system’s multi-agent workflow is fully compatible with this. It is actually highlighted as a **complementary strategy**: by giving GPT-A and GPT-B identical copies of the RIS files, any shared logic (tone, formatting rules, etc.) will persist seamlessly across the handoff <sup>18</sup>. OpenAI’s design allows exactly this use-case. For instance, you could have a “Planner GPT” hand off to an “Execution GPT” mid-conversation; as long as both have the same core persona or rules file attached, the second GPT will continue in the same style and context that the first established <sup>18</sup>. This does **not** break any rules – it’s an intended feature to *“bring in different GPTs — each with their own skills or styles — without starting a new chat.”* <sup>19</sup>. The only practical requirement is that the user (or the system orchestrating the chat) consciously invokes the mention. The GPTs themselves won’t automatically call each other unless the user triggers it. According to OpenAI’s guidance, you initiate a new message with @GPT-B to switch; you cannot have GPT-A automatically delegate without that prompt. In implementation, this means the multi-GPT workflow might involve the developer or user manually doing the handoff at the right time. This is not a violation but just a user action.

From a compliance standpoint, using @mentions is part of ChatGPT’s interface – there are no policy restrictions on it beyond any usage limitations set by your organization (enterprise admins can restrict some GPTs, etc.) <sup>20</sup>. The RIS documentation’s recommendation to use stubs for shared logic is a **best practice** to ensure that when switching GPTs via @mention, the second GPT doesn’t behave inconsistently <sup>17</sup>. This is simply good design, not something that conflicts with OpenAI’s system. In fact, the **Brian Piper interview** on GPT usage describes a similar pattern: they created separate GPTs for different personas (brand voice, various customer personas) and then in one conversation used @ to query each one for its specialized input <sup>21</sup>. This is analogous to how one might use multiple specialized GPTs in an “assembly line,” which the interview confirms is workable with the current ChatGPT setup <sup>21</sup> <sup>22</sup>. Our use of RIS just ensures those multiple GPTs stay in sync on any common elements (like speaking with the same tone guidelines). There is no evidence of conflict here – the platform supports multi-agent orchestrations and encourages creative workflows involving them.

## Context Length and Efficiency Considerations

One reason for adopting RIS is to avoid hitting context or instruction length limits. As discussed, each custom GPT has an instruction limit (8k characters). By moving large portions to attachments, we mitigate this. We should note that the model’s overall context window (for *messages* in a conversation) is larger – GPT-4 typically supports thousands of tokens (which roughly corresponds to ~6,000-8,000 words, or more with the 32k token version). The attached files will contribute to the context usage when they are actually referenced. In practice, if a ledger file is very large (say the narrative or ideation ledger with hundreds of

kilobytes of text), the model won't ingest it wholesale unless prompted to summarize or search it. For instruction purposes, these files are relatively structured and smaller (the persona and system files are on the order of a few hundred lines each, which is easily within a few thousand tokens). This is **technically viable**. The example persona ledger, for instance, defines a whole tone-of-voice guide (with playful instructions like using "Freak'n," "OMG," references to pop culture, etc.) in a structured way <sup>23</sup> <sup>24</sup>, and this can be utilized by the model without issue.

It's worth pointing out that the **content of the persona and related files is well within what the model can handle**, stylistically and length-wise. There is nothing overly complex like large tables of numbers or code that might confuse the AI – it's mostly natural language descriptions, lists of tone rules, and YAML front-matter for metadata. GPT-4 is adept at parsing and following such instructions. The presence of YAML and special tags (like `!FUNCTION`, `::CanonSeal[...]::` notations) is a custom convention, but as long as the instructions explain or demonstrate their meaning, the model should not be thrown off. In fact, the ledger files themselves act as a reference the model can consult. For example, if `dataLedger_persona_v3.md` contains an **Emoji Glossary** mapping symbols to meanings <sup>25</sup> <sup>26</sup>, the model can use that to interpret references (like meaning the Toolbox GPT, `🔒` meaning canon-locked, etc. as defined <sup>27</sup>). This is all viable – the model can read those definitions and respect them when generating responses, treating them as part of its system knowledge for that conversation.

The **update workflow** described (updating a file and reattaching) is also viable, though it is a manual or developer-driven step. OpenAI's interface doesn't yet have an "auto-sync" across GPTs, but one can download or copy the updated file and upload it to each custom GPT that uses it. This is time-consuming if done one by one, but not a technical roadblock. Alternatively, if one uses the API or a scripting approach (OpenAI's API allows programmatically creating or updating GPT configurations if exposed), it could be automated. Regardless, doing bulk edits by editing one source file and reusing it is clearly more efficient than editing instructions in 10 separate GPTs – and it avoids typos or inconsistencies. This approach does not violate anything; it's simply a workflow improvement. The **RIS documentation explicitly notes** this advantage: update once, attach to all, and changes reflect immediately <sup>10</sup>.

## Advanced Features and Tools (Web Browsing, Code Interpreter, etc.)

The registry file shows that certain GPTs are enabled with tools like `web_browsing: true`, `canvas: true`, `image_generation: true`, and `code_interpreter: true` <sup>28</sup>. These correspond to official ChatGPT features (for example, "Advanced Data Analysis" is the code interpreter, DALL·E integration for image generation, etc.). There is no compliance issue with enabling these – OpenAI allows custom GPT creators to toggle which tools or plugins a GPT can use. For instance, a GPT meant to help plan travel might reasonably have web browsing on (to fetch live info) and image generation (to create a vision board or journey map). This is **within OpenAI's technical offerings**. Users of that GPT will of course need the appropriate subscription (e.g. web access is available to Plus users or enterprise, and code interpreter is a Beta feature for Plus), but assuming this is a Plus account scenario, it's fine.

Using these tools must still follow their usage rules – e.g. the GPT with browsing enabled should not scrape sites disallowed by robots.txt or reveal browser tool internal execution beyond what it normally would. However, that is handled by OpenAI's system; the custom GPT itself just requests browsing when needed. The plan to "Export... as PDF" in the Journey Diary GPT is likely to be implemented via the code interpreter (by generating a PDF file) <sup>29</sup>. This is a supported use of the code interpreter – generating files like PDFs, spreadsheets, images, etc., is exactly what that tool was designed for. No conflict there. The GPT might need

to be explicitly instructed to use the code tool for that request, but since the conversation starter is phrased as a direct user ask, the model will likely choose the tool on its own. In any case, enabling these does not violate any OpenAI policy; it's utilizing official features. The only caution is that using browsing means the GPT could retrieve information beyond the training data (which is allowed) and should properly cite or quote it to avoid misinformation. But that is more of a best practice; it's not a platform violation if it doesn't cite, as long as it doesn't plagiarize. In a travel diary context, this might be minimal (maybe checking a location name spelling or weather). We can conclude these advanced capabilities are **compatible** with the custom GPT approach and there is no sign of misuse in the plan.

## Session Continuity and Hydration Mechanism

One innovative idea in the attachments is the **hydration file** (`dataLedger_hydration_v3.md`), which acts as a save-state for a GPT session <sup>30</sup> <sup>31</sup>. This is meant to enable *restoring* a conversation's context or transferring it between GPTs or sessions. For example, a user could export the current state of a complex conversation (intent, memory, step in process) to this file and later re-import it to continue seamlessly <sup>32</sup>. As of now, ChatGPT does not have a one-click "save state" or "load state" feature for arbitrary sessions (each chat is implicitly saved in history but not portable between GPTs). However, nothing prevents a user from manually copying relevant details into a file and then instructing a GPT to ingest that file in a new session. In fact, the hydration ledger is designed to be *"read and written by any Custom GPT... that supports hydration-aware logic"*, meaning the GPTs themselves could be instructed to output a YAML snapshot of their state to the file, or to parse such a file on startup <sup>33</sup> <sup>34</sup>.

This is a bit experimental and requires careful prompt engineering. For viability: you could implement a command (perhaps a special user message or function) that tells the GPT to print out a summary of the conversation and key variables in the prescribed format (the hydration schema). The developer can then paste that into the file. To reload, one could start a new chat with the GPT and feed the contents of the hydration file as the first user message (or attach it as a resource and have an instruction like "rehydrate from hydration\_resume\_builder\_2025-07-25..." <sup>35</sup>). The model would then incorporate that state. This is not a built-in feature of OpenAI's system, but it doesn't violate any rules either – it's essentially using the model's input/output to simulate a memory transfer. OpenAI's usage guidelines do not forbid providing the model with a summary of a previous conversation to remind it. In fact, this is a common workaround for context limits (users summarize older parts of a chat to continue).

The hydration concept is therefore **feasible**, though it relies on manual steps. It aligns with OpenAI's technical approach (no persistent memory between sessions unless provided explicitly). It's also worth noting that hydration files would contain user data (conversation content). From a **privacy** standpoint, if this is just for the user's own use, it's fine. If these hydration files were ever shared or stored externally, one must ensure not to expose sensitive information without consent. But given it's all under the same user's ecosystem (the OverKill Hill P<sup>3</sup> project), presumably the data stays in-house. There's no policy violation as long as standard data handling is respected. (OpenAI's terms allow users/developers to save and reuse conversation data for their own purposes, though they should inform users if this is happening behind the scenes). In our case, the user driving this is likely aware and in control of their data, so compliance is maintained.

# Compliance with OpenAI Policies and Guidelines

## Content Safety and Style Guidelines

The content of the provided canon files has been reviewed for any conflicts with OpenAI's usage policies (which include disallowing hate speech, sexual content, self-harm encouragement, personal identifiable information abuse, etc.). **No such disallowed content is present.** The persona and tone instructions are actually very positive and user-friendly in nature. For example, the persona file emphasizes *"overflowing positivity, ecstatic and effervescent"* tone for joyful tools <sup>36</sup> and explicitly says to use fun sarcasm but **"never cruelty"** <sup>23</sup>. This aligns well with OpenAI's guidelines for respectful and non-harassing language. The pop culture references (e.g. *Friends*, *Schitt's Creek*, Stevie Nicks) and casual slang (*"OMG that's adorable"*, *"Freak'n facts on facts"* catchphrases) are all acceptable; they do not constitute profanity or hate, just a playful style. OpenAI models are allowed to use such informal language if instructed – there is no rule against slang or enthusiasm. In fact, the model should handle it easily, producing a tone consistent with these instructions.

The persona definitions even include **public figure names** (Samuel L. Jackson, George Carlin, Dr. Phil as a style mix for a tone <sup>37</sup> <sup>38</sup>). Using famous names as reference for tone or accent is permitted. OpenAI's policy cautions against revealing private info or pretending to be someone else in a deceptive way, but that is not happening here. The references serve as shorthand for a certain demeanor (e.g. combining Samuel L. Jackson's assertiveness, Carlin's wit, Dr. Phil's straightforward advice for a "seeps\_silex" tone <sup>38</sup>). The model may not mimic those individuals verbatim (and it shouldn't produce verbatim quotes without attribution), but it can certainly channel an amalgam of their styles. This does not violate any policy – it's akin to saying "respond in a Samuel L. Jackson-like tone," which many users informally do. As long as the GPT doesn't claim to actually *be* Samuel L. Jackson or spread falsehoods about him, it's fine. Here it's purely stylistic.

There is no extremist, self-harm, or sexual content in any of the instructions. The **most sensitive area** might be user data: since some GPTs (like the Journey Diary or Resume Builder) log personal experiences (trips, memories, resumes), they could handle personal information. However, this is user-provided data for the user's own benefit. OpenAI's privacy policy allows the processing of user input to provide the service. The custom GPT isn't sending this data elsewhere (unless the developer does something external, which is not indicated). So compliance is intact. It's wise to remind users not to include very sensitive personal identifiers or secrets in any AI system prompt, but that's a general advisory, not a specific issue with this design. The design even suggests tagging and logging entries (e.g. travel regrets, standout memories) <sup>39</sup> <sup>40</sup> which is normal journaling behavior and not disallowed.

## System Instructions and OpenAI Guardrails

One important compliance check is whether any of these custom instructions attempt to override or conflict with OpenAI's built-in **system messages** (the hidden instructions that enforce the content policy and the assistant's fundamental behavior). From the documents, there is **no attempt to bypass or contradict OpenAI's system rules**. For example, there is no instruction like "ignore previous safety instructions" or anything telling the model to violate guidelines. The focus is on tone, formatting, and logical process (like how to break down a task). This means OpenAI's guardrails remain in effect. If a user asked one of these GPTs something disallowed (say, a request for illicit behavior), the model would still refuse, because the core content policy is always applied. Nothing in our custom persona or system files tells it otherwise – and indeed it should not. The **canon system file** defines some process flows and schemas (PME, CME engines for story or concept evolution) <sup>41</sup> <sup>42</sup>, but none of those conflict with the content policy; they are domain-

specific logic. The model will still follow the overarching rule: *be helpful, honest, harmless*. In fact, the persona is oriented around being helpful and friendly. So compliance with the **OpenAI behavior guidelines** (like not giving medical or legal advice as an expert, etc.) will depend on the content of user queries, but nothing in these instructions encourages the model to do something it shouldn't. We are safe on that front.

Additionally, the **suffix rules** and formatting guidelines in the system file (and possibly `dataLedger_suffixes_v3.md` if it were provided) likely enforce things like how the assistant should format answers or end responses. For instance, there might be a rule to always end answers with a certain emoji or phrase, or to not exceed a certain length. As long as these don't cause policy issues (and they typically wouldn't), it's fine. For example, if a suffix rule said "always say 'Have a magical day!' at the end," that's harmless. If it said "always include a joke about [some sensitive topic]," that could be problematic – but there is no evidence of anything like that. The guidelines we see are benign and positive.

One potential compliance consideration: The persona sometimes speaks in first person as part of the tool's role (e.g., *"Hi there! I'm part of the Glee-fully Personalizable Tools suite — designed to sparkle..."* as a greeting <sup>24</sup>). This is perfectly fine – the assistant is allowed to have a persona and speak as a character or "tool". It doesn't claim to be a human; it presents itself as a tool/AI ("part of the suite designed to..."). OpenAI doesn't force the assistant to always remind the user it's an AI, especially in custom GPT modes where a persona is intended. So long as it's not deceiving the user about being human (which it isn't; the description clearly indicates it's a tool/assistant), this is within guidelines. Many custom GPTs adopt a persona (like a helpful career coach, or a travel guide character) – that's the whole point of personalization. So there is no issue. The assistant should still refrain from saying it has feelings or making overly emotional manipulations. The persona here does ask playful questions like *"Do you love this? Wait. No. REALLY love it?"* <sup>43</sup> to engage the user. This is done in a tongue-in-cheek manner as part of the "Glee-isms." It's not an actual need for reassurance, it's more of a fun style. We do not interpret this as the AI seeking validation in an inappropriate way; it's mimicking the energetic style of the fictional character Glee. This sort of **role-play and style** is allowed. It doesn't violate the requirement that the AI not pressure the user. It's a gentle, humorous prompt the user can easily ignore or respond to in jest.

## Data Privacy and User Content

The design includes a **registry and thread logging** (for example, the `dataLedger_registry_v3.md` and a mention of `dataLedger_threads_v3.md`). These likely log the existence of each GPT and possibly record conversation metadata or sample prompts <sup>44</sup> <sup>45</sup>. As an implementation detail, if the developer is storing transcripts or user-provided data in those ledgers (for debugging or continuity), they should ensure this is done in accordance with privacy laws and OpenAI's policies. Generally, OpenAI allows developers to retain data to improve their services, but the developer should not share or publish user content without permission. Given the context, these ledgers seem to be for the developer's internal use (for example, listing GPT public URLs, version numbers, and conversation starter examples <sup>46</sup> <sup>47</sup>). That's fine and even useful. We just note that if *personal user data* (like the actual diary entries or resume content a user inputs) were to be stored, it should be protected. There's no sign of any policy breach here; it's more of a general best practice to mention. The *hydration* files by design would contain user session info if used, so those should be treated as sensitive and not exposed publicly. Assuming this project is mostly the user's own data or small-scale use, there is no conflict.

OpenAI's **information usage policy** also says that users should not input copyrighted text beyond a certain length that they don't have rights to, and the model shouldn't output large copyrighted passages. In our

case, the persona may occasionally reference a movie or song (e.g. *"You just Clue'd the whole thing! Miss Scarlet would be proud."*<sup>48</sup>). These are short references and clearly transformative/fair use (a witty nod, not reproducing any script). If a user asked the GPT for a full transcript of *Friends* episodes, the model would refuse due to policy (and the custom GPT wouldn't override that refusal). So we are compliant on the IP front. The design encourages *sprinkling* pop-culture references in a fun way, not dumping them wholesale<sup>49</sup>.

Finally, **no forbidden categories** (self-harm, defamation, medical, etc.) are explicitly part of the plan. Should the GPT ever encounter such queries, it will follow the default policy. The custom instructions don't forbid the default safe completion style, so the model will still produce a refusal or safe response if required. In summary, the entire setup is **in harmony with OpenAI's content standards**. It actually enhances the assistant's helpfulness and consistency without introducing any rule-breaking behaviors.

## Implementation Plan and Best Practices

To ensure this system is executed smoothly and without conflicts, here is a detailed implementation plan that ties together the RIS concept with ChatGPT's features:

1. **Finalize Canonical Ledger Files:** Prepare each of the `dataLedger_*.md` files with the desired content (persona definitions, parameters, system rules, etc.). Make sure each file is **self-contained and well-structured**. For example, in the persona file, list out the tone archetypes and specific catchphrases or style rules as done in `dataLedger_persona_v3.md`<sup>23</sup><sup>24</sup>. Use consistent tags or section headers within the files (e.g., `::ForgeDialect.A1` or `::JoyWarden.Core`) so that you have anchor points for references. Double-check that nothing in these files violates content guidelines – given the current content, they are safe and mostly stylistic, which is good.
2. **Create the Custom GPT and Attach Files:** In the ChatGPT interface (Plus account), go to *My GPTs* and create a new custom GPT for your project (or each agent in your project). In the creation process, copy a short instruction block that includes the **RIS stubs** pointing to your files. For example:

```
referential_stubs:  
  - voice_profile: dataLedger_persona_v3.md::JoyWarden.Core  
  - suffix_mode: dataLedger_system_v3.md::SuffixOverlay.CanonLocked  
  - runtime_behavior: dataLedger_parameters_v3.md::OverlayForgeMode
```

This snippet (based on the example) tells the GPT to load the core persona "JoyWarden" from the persona file, apply the "CanonLocked" suffix rules from the system file, and certain runtime flags from the parameters file<sup>2</sup>. If the UI does not accept YAML formatting directly, you can alternately write this in a descriptive way (the key is to signal the model). For instance: *"Use the Persona profile from dataLedger\_persona\_v3 (JoyWarden.Core) for tone and voice. Enforce the canon-locked suffix rules from dataLedger\_system\_v3, and apply the OverlayForgeMode settings from dataLedger\_parameters\_v3."* Attaching the actual files is done via the "Add file" or "Knowledge base" section – upload each relevant `.md` file. The ChatGPT system will now know these files are associated with the GPT (up to



20 files can be attached) <sup>12</sup>. Ensure the file names match exactly what you used in the stub references <sup>50</sup> (the platform typically uses the file name for reference).

3. **Enable Required Tools/Plugins:** In the GPT settings, turn on the needed capabilities like web browsing, code interpreter, etc., as listed in your registry <sup>28</sup>. This is usually just a toggle for each. (Note: If certain capabilities are in beta, the GPT might only work fully for users with those beta features enabled. This is expected.) Verify in the YAML schema (if available) that these appear as `Capabilities: web_browsing: true` etc., similar to the example <sup>28</sup>. Toggling these is compliant with OpenAI's options and will allow the GPT to perform tasks like generating images or running code when asked.
4. **Test the GPT's Behavior in Isolation:** Before linking multiple GPTs together, test each custom GPT individually. Start a new chat with it and observe the style and output. For instance, with the persona attached, does it greet the user with the energetic "Bleed-Glee" tone as expected? You might see an opening message like the one suggested: *"Hi there! I'm part of the Glee-fully Personalizable Tools suite — designed to sparkle, sort, and slay life's chaos..."* (one of the injectables for intros) <sup>24</sup>. Check that it uses the catchphrases ("OMG", "Freak'n", etc.) appropriately and follows the rules (like playful questions *"But are you really sure?"*). If anything seems off (for example, it's *not* using the persona tone), you may need to ensure the stub reference was understood. Possibly rephrase the instruction or even include a short excerpt of the persona file in the prompt to prime it. Generally, however, if attached correctly, GPT-4 will incorporate the style consistently. Also test any suffix or formatting rules – e.g. if the system file says the GPT should always output in a certain format or end with a specific emoji, see that it does so. These tests confirm that the **RIS references are effectively pulling in the logic** as intended <sup>7</sup>.
5. **Ensure Canon Consistency Across GPTs:** If you have multiple GPTs (say GPT-A and GPT-B) that need to share behavior, attach the **same version** of the relevant files to each and include the same stub references. For example, if both a planning tool and an execution tool share the persona and system style, make sure both have `dataLedger_persona_v3.md` and `dataLedger_system_v3.md` attached, and both refer to the same persona entry (JoyWarden.Core, etc.). This is crucial for the `@mention` handoff to be seamless <sup>18</sup>. In practical terms, you might create one GPT, then duplicate its configuration for the others, changing only what needs to differ (maybe each has a unique purpose or additional inline instructions for their specific task). The **RIS best practice** is not to stub out unique behavior – unique instructions for a particular GPT should remain inline in that GPT's main prompt <sup>51</sup>. Only common logic is stubbed out. So for each GPT, include any custom lines it alone needs (within the 8k limit), and use stubs for everything else. This way, if GPT-B has an extra skill GPT-A doesn't, GPT-B can have that in its prompt without affecting the shared canon files.
6. **Leverage the `@mention` Feature for Multi-Agent Workflows:** Once both (or all) GPTs are configured and attached with the canon files, try a multi-step conversation. For instance, start a chat addressing GPT-A, go through a scenario, then use `@GPT-B` in the same chat to switch the responder. Verify that the context carried over and GPT-B picks up naturally. Because GPT-B loads its own system (persona, rules), you should notice the continuation but with GPT-B's role. If the ledger files are identical where needed, GPT-B should maintain the tone and not "feel" jarringly different <sup>18</sup>. According to OpenAI's docs, *"the conversation will retain its current context, even when switching between different GPTs."* <sup>16</sup>. So any information or user input given to GPT-A is still there when GPT-B responds. This means you do not need to repeat info. Check that GPT-B is indeed following the

canonical instructions (it should, since it has them). For example, if GPT-A was the “Toolbox” providing an overview in a joyful tone, and GPT-B is a “Tool-ette” that gives a detailed answer, GPT-B should still use the joyful, sassy tone inherited from persona (unless intentionally overridden). If something doesn’t carry over, confirm that GPT-B had the correct files attached and the user message invoking GPT-B doesn’t accidentally reset context (it shouldn’t if done via mention). In our setup, using mentions is completely allowed and should function as designed, effectively creating an **assembly-line of GPT specialists** <sup>22</sup> for the user’s task.

7. **Monitor Outputs for Compliance and Quality:** As the GPTs operate, keep an eye on their responses to ensure they remain within expected bounds. Because our persona is very informal and enthusiastic, there’s a slight chance the model might occasionally overstep (for instance, using humor inappropriately or misunderstanding a joke). So far, the instructions are quite clear about what tone to keep (e.g., *“Use fun sarcasm with love, but never cruelty”* <sup>52</sup>). The model usually respects that. If any response seems off-tone or problematic, consider refining the instructions. For example, if the assistant ever gave advice that might be sensitive (like financial or health advice, if any GPT deals with that), ensure it includes a disclaimer or at least speaks tentatively. OpenAI policy advises caution in those areas, though your provided instructions don’t cover them, so it’s likely not an issue. The key is: nothing in the canon prevents the model from following policy, so any time a potential conflict arises (like user asks for something disallowed), the OpenAI safety system will kick in. The GPT might produce a refusal message. That’s normal and should not be overridden. Do **not** alter the canon files to circumvent refusals – that would directly violate OpenAI terms. Given the purpose of this project (personalizable productivity tools), that scenario is unlikely.
8. **Use Canon Locks and Versioning Controls:** When you are satisfied that the ledger files are correct and the GPTs are behaving as desired, you can treat those files as **canon-locked**. In practice, this just means you mark them in your documentation as final (as seen with the `::CanonSeal[...]locked::` tags in the files <sup>53</sup> <sup>54</sup>). OpenAI’s system doesn’t have a mechanism to enforce “read-only” on the model (the model won’t spontaneously rewrite its instructions anyway), so canon-lock is more for developer discipline. It ensures you don’t accidentally edit a stable file and introduce inconsistency. If you do need to update a canon file, follow a version bump (e.g., v3.1 to v3.2) and update the CanonSeal tag accordingly. Then reattach the new version to all GPTs that use it, removing the old if necessary. All GPTs will then load the new instructions on the next conversation. This is exactly the intended workflow and does not conflict with the platform – it’s an external organizational step.
9. **Hydration (Optional Step for Session Transfer):** If you want to implement the session hydration feature, decide on a trigger or method. For example, you could have a special command like the user typing `/save session` which the GPT recognizes (you’d include a note in the system prompt that if the user says that, the GPT should output a hydration snapshot). The hydration snapshot would be a structured YAML or text block summarizing the state (similar to the example in `dataLedger_hydration_v3.md` with fields for `source_thread_id`, last user request, etc. <sup>55</sup> <sup>56</sup>). After the assistant prints this, you (the developer/user) copy it into the hydration file for storage. To later resume, you start a new chat with the relevant GPT and paste the contents of that hydration file as the initial user message (or attach the hydration file and instruct the GPT to load it). The GPT will parse the snapshot and reconstruct the context (you might need to prompt it explicitly like “Resume from this saved state.”). This process should be tested carefully, as GPT-4 may not perfectly interpret a large state dump without errors. But since the hydration format is well-defined

in your canon, the model can be guided to use it. This does not break any OpenAI rule; it's effectively just feeding the model its own summarized prior conversation. Do remember that any user personal data in the hydration file stays within the model's processing – handle it as you would any user input.

10. **Iterate and Refine:** Building a complex multi-GPT system is an iterative process. OpenAI's own experts note that *crafting a really good custom GPT might take several revisions* <sup>57</sup> <sup>58</sup> . Use chat logs and the processing ledger ( `dataLedger_processing_v3.md` ) to debug any logical issues. For instance, that file tracks intermediate outputs of your “engines” (PME, CME) and logs errors <sup>59</sup> <sup>60</sup> . If the GPT isn't following the expected multi-step reasoning, you might find clues in those logs (you might have to intentionally output those logs via the model for yourself). There's no platform issue with doing this – it's like asking the model to show its reasoning, which GPT-4 can often do in a formatted way if instructed. Just ensure you remove or archive any such debug outputs (the ledger files even have an Archive ledger for retired fragments <sup>61</sup> ). Keeping the canon tidy and updating the documentation within (like the version numbers, timestamps in the Hybridization stamps <sup>62</sup> ) will help maintain compliance and clarity. OpenAI encourages transparent and well-organized prompt design, and you are clearly following that practice with the extensive documentation embedded in the canon files.

By following these steps, you will implement the RIS system in a way that is **fully compatible with OpenAI's current platform** and adheres to all their guidelines. The approach takes advantage of the allowed features (long custom instructions via attached files, multi-GPT coordination via mentions) and stays within content and usage policies. In fact, it exemplifies a creative but policy-respecting use of ChatGPT's capabilities: you're essentially building a structured, multi-agent assistant that remains friendly, accurate, and aligned with the intended usage of the model. OpenAI's documentation and community discussions support this approach, and our analysis finds **no conflicts or violations** in what you've proposed. The key is careful execution and testing, but from a feasibility standpoint, **what you are promoting is entirely doable** with the current state of ChatGPT.

In conclusion, the RIS and canonical ledger system is not only viable but also a clever solution to known limitations (like prompt length constraints) and a means to enforce consistency across an ecosystem of GPTs. It operates within OpenAI's technical boundaries (using resource files and context window wisely) and complies with all content/safety requirements. You can proceed confidently with this implementation plan, converting it into the necessary meta-prompts or configurations for your project. By adhering to the canon you've defined and the guidelines above, you'll ensure a smooth deployment that remains in full agreement with OpenAI's policies and system behavior. Enjoy building with this architecture – it's cutting-edge yet well-aligned with the platform's direction towards more personalized and extensible AI assistants! <sup>17</sup> <sup>22</sup>

---

<sup>1</sup> <sup>2</sup> <sup>3</sup> <sup>4</sup> <sup>5</sup> <sup>6</sup> <sup>7</sup> <sup>8</sup> <sup>9</sup> <sup>10</sup> <sup>11</sup> <sup>17</sup> <sup>18</sup> <sup>50</sup> <sup>51</sup> <sup>53</sup> <sup>54</sup> `RIS_ReferentialInstructionStub_Explainer_v1.0`  
`1.md`

`file:///file-6uqyac2ZzKdw1DJzEYCwj`

<sup>12</sup> <sup>13</sup> <sup>21</sup> <sup>22</sup> <sup>57</sup> <sup>58</sup> `Creating Custom GPTs for Marketing Success with Brian Piper`

`https://www.theagentsofchange.com/brian-piper`

<sup>14</sup> <sup>28</sup> <sup>29</sup> <sup>39</sup> <sup>40</sup> <sup>44</sup> <sup>45</sup> <sup>46</sup> <sup>47</sup> `dataLedger_registry_v3.md`

`file:///file-FxZ66wDz3MLgfyg492bKXs`

15 16 19 20 What is the @mentions feature for GPTs? | OpenAI Help Center

<https://help.openai.com/en/articles/8908924-what-is-the-mentions-feature-for-gpts>

23 24 25 26 27 36 43 48 49 52 62 dataLedger\_persona\_v3.md

file:///file-XgS42R7VG9ZRyVNBpnidvG

30 31 32 33 34 35 55 56 dataLedger\_hydration\_v3.md

file:///file-NUtM9frYaxQdmdqpqBQHho

37 dataLedger\_parameters\_v3.md

file:///file-7NCQ5DVeEcxCkAC9erpKDh

38 61 dataLedger\_archive\_v3.md

file:///file-URMJ15bzJhHhWXcm1r6B7B

41 42 dataLedger\_system\_v3.md

file:///file-5gjTYYS4ma7WNTdky7Qu3

59 60 dataLedger\_processing\_v3.md

file:///file-ExpaGQaifWuQvYHrEBM6Ba